

Upgrades

Kubernetes Production · Lesson 17

Graceful Kubespray upgrades + universal version-skew rules

kubespray.io · kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Run a **graceful, rolling upgrade** with `upgrade-cluster.yml` (cordon → drain → uncordon)
- Follow the workflow: **checkout matching tag** → **bump kube_version** → **run** → **verify**
- Apply the **version-skew rule** — one minor at a time, control plane before workers
- Control the roll-out with `serial`, `--limit`, and the `upgrade_node_*` vars
- Recognise the **unsafe** `cluster.yml -e upgrade_cluster_setup=true` path
- Explain the **bridge**: Kubespray runs `kubeadm upgrade` under the hood

Reference: *Upgrading Kubernetes in Kubespray* — kubespray.io · and
Upgrading kubeadm clusters — kubernetes.io/docs

17.1 Graceful Upgrades with `upgrade-cluster.yml`

The Graceful Upgrade Playbook

`upgrade-cluster.yml` (note the **exact hyphenated name**) is the **graceful, serial** upgrade path for a running Kubespray cluster.

- Unlike `cluster.yml`, it **cordons** → **drains** → **uncordons** each node during a cluster upgrade, so workloads are evacuated **one batch at a time** instead of all at once.
- Requires **at least one** `kube_control_plane` **already deployed** — it upgrades an existing cluster, it does **not** bootstrap one.
- Driven by **one declarative knob**: `kube_version`, the target Kubernetes version.

`cluster.yml` re-applies state but does **not** drain — only
`upgrade-cluster.yml` evacuates workloads safely batch by batch.

Cordon / Drain / Uncordon — One Batch at a Time

For every node it touches, the playbook performs the safe maintenance cycle:

Step	What happens
Cordon	Mark the node unschedulable — no new Pods land
Drain	Evict workloads gracefully, respecting PodDisruptionBudgets
Upgrade	Bump packages, run <code>kubeadm upgrade</code> , restart the kubelet
Uncordon	Re-enable scheduling once the node is back on the new version

- The default batch size is **20%** of available nodes; tune it with `serial`.
- This is the **same cordon/drain you learned in CKA node maintenance** — Kubespray just orchestrates it across the whole fleet.

17.2 The Required Workflow

Workflow: tag → kube_version → run → verify

The upgrade is always four ordered steps:

1. **Check out the matching Kubespray release tag** — one tag at a time.
2. **Bump kube_version** in `group_vars/k8s_cluster/k8s-cluster.yml` (or pass `-e kube_version=vX.Y.Z`).
3. **Run** the graceful playbook.
4. **Verify** between steps.

```
ansible-playbook upgrade-cluster.yml -b \  
-i inventory/sample/inventory.ini \  
-e kube_version=v1.31.4
```

```
kubectl version          # control plane on the new version?  
kubectl get nodes        # every node Ready on the new build?
```

Check Out the Tag, Bump the Version

The **Kubespray tag and the Kubernetes version are coupled** — each tag targets a specific K8s range, so move **one tag at a time**.

```
# 1. Match the Kubespray release tag to your target step
git checkout v2.24.0
```

```
# 2. group_vars/k8s_cluster/k8s-cluster.yml
kube_version: v1.31.4
```

Variable **formats and locations change between Kubespray releases** — diff the upstream sample inventory for the new tag **before** running, or the upgrade can fail on drifted vars.

17.3 Version-Skew: Never Skip a Minor

One Minor at a Time, Control Plane First

This rule is **identical** in the CKA kubeadm course and the Kubespray docs — it is a property of **Kubernetes/kubeadm**, not of any one tool.

- **Upgrade one minor version at a time; never skip a minor.** Patch jumps within a minor are fine.
- **Order: control plane first, then workers** — primary control plane → additional control planes → worker nodes.

Kubespray tags	Kubernetes minors
v2.22 → v2.23 → v2.24 ✓	1.30 → 1.31 → 1.32 ✓
v2.22 → v2.24 ✗ (skips a minor)	1.30 → 1.32 ✗ (skips a minor)

"Upgrading from an older release straight to the latest is unsupported and likely to break something." Kubespray respects kubeadm's version-skew rules — because it **is** kubeadm underneath.

17.4 Controlling the Roll-out

serial · --limit · upgrade_node_*

Control	Effect
(default)	Upgrade in batches of 20% of available nodes
<code>-e serial=1</code>	Upgrade N nodes at a time (here, one — safest)
<code>--limit "node4:node6"</code>	Restrict to specific nodes only
<code>upgrade_node_confirm: true</code>	Manual approval before each node
<code>upgrade_node_pause_seconds: 60</code>	Automatic pause before each node
<code>upgrade_node_post_upgrade_confirm: true</code>	Manual approval after each node
<code>upgrade_node_post_upgrade_pause_seconds: 60</code>	Automatic pause after each node

The `upgrade_node_*` confirm/pause vars let you gate each node with a human checkpoint or a fixed delay — invaluable for cautious production roll-outs.

serial=1 and --limit Examples

```
# One node at a time (safest, slowest)
ansible-playbook upgrade-cluster.yml -b \
  -i inventory/sample/inventory.ini \
  -e kube_version=v1.31.4 -e "serial=1"
```

```
# Refresh facts first, then upgrade only named nodes
ansible-playbook playbooks/facts.yml -b \
  -i inventory/sample/inventory.ini
ansible-playbook upgrade-cluster.yml -b \
  -i inventory/sample/inventory.ini \
  -e kube_version=v1.31.4 --limit "node4:node6:node7:node12"
```

Always run `playbooks/facts.yml` before a `--limit` run so Ansible has **fresh facts** for the whole cluster and does not act on stale data.

17.5 Graceful vs. Unsafe

The Unsafe Non-Graceful Path

There is a faster but **disruptive** alternative — know it so you avoid it.

```
# UNSAFE: bypasses graceful cordon/drain
ansible-playbook cluster.yml -b \
  -i inventory/sample/inventory.ini \
  -e kube_version=v1.31.4 \
  -e upgrade_cluster_setup=true
```

Path	Behaviour
<code>upgrade-cluster.yml</code> (recommended)	Cordon → drain → uncordon, one batch at a time
<code>cluster.yml -e upgrade_cluster_setup=true</code>	Migrates control plane immediately, no drain → workload disruption

Use the unsafe path **only** when you can tolerate disruption. For every production version bump, use `upgrade-cluster.yml`.

17.6 Under the Hood — the kubeadm Bridge

Kubespray Runs `kubeadm upgrade` Per Node

Per node, the graceful playbook does **exactly** what the manual CKA procedure does — same kubeadm mechanics, driven declaratively:

1. **cordon + drain** the node (respecting PodDisruptionBudgets)
2. upgrade the kubeadm/kubelet packages
3. run `kubeadm upgrade apply vX.Y.Z` on the first control plane, `kubeadm upgrade node` everywhere else
4. **restart the kubelet**, then **uncordon**

So the CKA *"kubeadm upgrade"* knowledge **maps directly here** — one minor at a time, control plane first, drain during the per-node upgrade, certs auto-renewed on `kubeadm upgrade`. Kubespray just runs it across the fleet from one `kube_version` change.

Key Takeaways

- `upgrade-cluster.yml` = graceful, serial upgrade — cordon → drain → uncordon, one batch at a time
- Workflow: checkout matching tag → bump `kube_version` → run → verify
- **Never skip a minor; control plane before workers** — `v2.22 → v2.23 → v2.24`, `1.30 → 1.31 → 1.32`
- Tune the roll-out with `serial` (default 20%), `--limit`, and the `upgrade_node_*` confirm/pause vars
- Run `playbooks/facts.yml` before any `--limit` run; diff the inventory between tags
- `cluster.yml -e upgrade_cluster_setup=true` is the **unsafe** non-graceful path — no drain
- Bridge: Kubespray runs `kubeadm upgrade` per node — same kubeadm mechanics, declarative

End of Lesson 17

Next: Lesson 18 — Backup, Recovery & Reset

```
git checkout v2.24.0 · upgrade-cluster.yml -e kube_version=vX.Y.Z · -e serial=1 · --limit
```